Exploring Monte Carlo Tree Search as a Reinforcement Learning Approach for Two-Player Games

Neha Aryasomayajula

1 Technical Motivation

As a casual chess enthusiast, I have been fascinated by the game's evolution over the past decade, driven largely by the rise of machine learning–based chess engines. For example, the King's Indian Defense was once considered a strong opening for grandmasters, yet modern engines have since disputed it. Among the most popular chess engines is Leela Chess Zero (Lc0), inspired by Google DeepMind's AlphaZero. Lc0 integrates Monte Carlo Tree Search (MCTS), a canonical reinforcement learning algorithm, with deep neural networks. MCTS is a particularly compelling research area in mathematical machine learning because it offers an efficient, inexact approach to planning in large state spaces where exhaustive search is infeasible.

2 Background on Monte Carlo Tree Search

In my project, I study Monte Carlo Tree Search (MCTS) for two-player games. MCTS identifies strong actions by iteratively building a search tree whose nodes represent game states. Each simulation starts at the root and selects child nodes according to a policy that balances exploration (trying unfamiliar moves) and exploitation (favoring actions with high estimated rewards). When the simulation reaches an unvisited state, the algorithm performs a random rollout to a terminal state to obtain an initial reward estimate. The node may be expanded immediately or, in some variants, only upon a later visit; either way, once expanded, its children become available for selection in future simulations. After each simulation, the resulting reward is backpropagated up the tree to update value estimates. Repeated simulations concentrate computation on promising branches while still maintaining exploration. Because the estimated rewards at each node depend on deeper parts of the tree that are gradually revealed, the induced decision process is non-stationary and difficult to analyze, raising questions about convergence rates, how regret scales with tree depth, and how to design effective exploration–exploitation policies.

3 Proposed Investigation

I will begin by reviewing foundational literature on MCTS, focusing on the formulation introduced in [3], which models tree search as a sequence of multi-armed bandit problems. This variant, known as UCT (Upper Confidence Bounds applied to Trees), employs the UCB1 selection policy to balance exploration and exploitation during search. Then, I will implement UCT and compare the resulting empirical performance with the theoretical results reported in the literature.

Standard UCB1 Selection Policy [1]:

$$U_j = \bar{X}_j + \sqrt{\frac{2 \ln n}{n_j}}$$

U_j	Upper confidence bound for move j
$\bar{X}_j$	Sample mean reward of move j
n	Total simulations from the current node
n_j	Simulations for move j

4 Does UCT Guarantee Convergence?

4.1 Convergence Proof (Sketch)

A finite-horizon Markov Decision Process is a model for sequential decision-making with a fixed number of steps. Kocsis and Szepesvári [3] prove that for finite-horizon Markov Decision Processes, the UCT algorithm is consistent: as the number of rollouts $n \to \infty$, the probability that UCT selects a suboptimal action at the root converges to zero at a polynomial rate. Moreover, the bias of the root-value estimate decays at a logarithmic rate:

$$O\left(\frac{\log n}{n}\right).$$

The difficulty in the analysis arises because UCT applies UCB1 at every node of the search tree. In a classical multi-armed bandit (MAB) problem, a player chooses repeatedly among several actions, each providing stochastic rewards from a fixed distribution, and the goal is to maximize cumulative reward by balancing exploration and exploitation. Unlike classical MABs, however, the payoff sequences observed at internal nodes of a search tree are non-stationary: as deeper parts of the tree are explored, the estimated values of child nodes change, inducing drift in the payoff distributions of their parents. The convergence proof shows that this drift is sufficiently controlled to preserve UCB1's concentration guarantees. My presentation simplifies results from sections 2.2–2.4 of [3].

4.1.1 Drift Conditions

At internal nodes of the UCT tree, the payoff distributions are not stationary: as deeper estimates improve, the expected payoff at a parent node changes over time. To analyze UCB1 in this setting, Kocsis and Szepesvári introduce drift conditions, which generalize the assumptions normally used in bandit analysis.
In the classical multi-armed bandit setting, each action produces a sequence of independent, identically distributed (i.i.d.) rewards. Hoeffding's inequality provides a bound on how far the sample mean of these rewards is likely to deviate from the true mean: for bounded variables $Y_1, \ldots, Y_s \in [0,1]$ with mean μ,

$$\mathbb{P}\big(\bar{Y}_s - \mu \geq \varepsilon\big) \leq \exp(-2s\varepsilon^2), \qquad \mathbb{P}\big(\bar{Y}_s - \mu \leq -\varepsilon\big) \leq \exp(-2s\varepsilon^2),$$

where $\bar{Y}_s = \frac{1}{s}\sum_{i=1}^{s} Y_i$. Intuitively, this inequality says that the probability of the sample mean deviating significantly from the true mean decreases exponentially with the number of samples. Let $\{X_{it}\}_{t\geq 1}$ be the payoff sequence for arm i in a potentially non-stationary setting, with $X_{it} \in [0,1]$. Define

$$\bar{X}_{is} = \frac{1}{s}\sum_{t=1}^{s} X_{it}, \qquad \mu_{in} = \mathbb{E}[\bar{X}_{in}], \qquad \mu_i = \lim_{n\to\infty} \mu_{in}, \qquad \delta_{in} = \mu_{in} - \mu_i.$$

The drift conditions require that:

1. **Convergence of means:** $\mu_{in} \to \mu_i$ as the number of samples grows.
2. **Concentration:** there exists a constant $C_p > 0$ such that

   $$\mathbb{P}\big(\bar{X}_{is} \geq \mu_i + c_{t,s}\big) \leq t^{-4}, \qquad \mathbb{P}\big(\bar{X}_{is} \leq \mu_i - c_{t,s}\big) \leq t^{-4},$$

   with

   $$c_{t,s} = 2C_p\sqrt{\frac{\ln t}{s}}.$$

To see the connection with the i.i.d. case, one can set the right-hand side of Hoeffding's inequality to t^{-4}, giving

$$\varepsilon = \sqrt{\frac{2\ln t}{s}}.$$

This corresponds to the drift condition with $C_p = 1/\sqrt{2}$ and $\delta_{in} = 0$: in other words, when rewards are i.i.d., the sample mean stays close to the true mean and the standard UCB1 exploration term $\sqrt{(2\ln t)/s}$ is sufficient to account for randomness.

In the non-stationary setting of UCT, however, payoffs can change over time as deeper parts of the tree are explored. Deviations of $\bar{X}_{is}$ then arise not only from random noise but also from the drift in expected payoff. The drift conditions require a slightly larger confidence term $c_{t,s} = 2C_p\sqrt{(\ln t)/s}$, which covers both sources of error. This ensures that UCB1 continues to assign sufficient exploration to each action even when the payoff distribution is non-stationary.

4.1.2 Supporting Theorems

Let $T_i(n)$ be the number of pulls of arm i by time n and u^* be the mean of the payoff distribution for the optimal arm. Under the drift conditions, generalized UCB1 analysis yields:

1. Suboptimal arms are pulled $O(\log n)$ times (Theorem 1).
2. Bias bound:
$$\left|\mathbb{E}[\bar{X}_n] - \mu^*\right| = O\left(\frac{\log n}{n}\right) \quad \text{(Theorem 2)}.$$
3. Exploration guarantee:
$$T_i(n) \geq \rho \log n \quad \text{for some } \rho > 0 \quad \text{(Theorem 3)}.$$
4. Concentration under drift: The sample average concentrates around its (possibly drifting) expected value (Theorem 4). Combined with Theorems 2 and 3, this ensures drift conditions propagate from children to parents.
5. Failure probability:
$$\mathbb{P}(\text{UCB1 selects a suboptimal arm at time } t) \leq Ct^{-\alpha} \quad \text{(Theorem 5)}.$$

4.1.3 Main Inductive Argument

We prove by induction on the horizon D that UCT generates payoff sequences at every node satisfying the drift conditions. Combined with Theorems 2 and 5, this implies consistency and an $O(\log n/n)$ bias bound.

Base Case ($D = 1$). If the horizon is 1, each action leads immediately to a terminal reward. The payoffs are i.i.d., so Hoeffding's inequality ensures that the drift conditions hold with $C_p = 1/\sqrt{2}$ and $\delta_{in} = 0$. In this case, UCT reduces to UCB1, and Theorems 2 and 5 apply directly.

Inductive Step. Assume that all nodes at depth $D - 1$ or greater produce payoff sequences satisfying the drift conditions. Consider a tree of depth D and focus on the root. The root's payoff for an action is the immediate reward plus the value of the resulting child state. While immediate rewards are i.i.d., the child values evolve over time as their subtrees are explored, creating non-stationarity at the root. By the induction hypothesis, each child's payoff sequence satisfies the drift conditions. Theorem 3 guarantees that every action at the root is sampled $\Omega(\log n)$ times, ensuring sufficient exploration. The parent node then satisfies the drift conditions as follows:

1. **Convergence of means:** Since child value estimates converge by the induction hypothesis, the parent's expected payoff also converges. Theorem 2 shows this convergence occurs at rate $O(\log n/n)$.
2. **Concentration bounds:** Although the parent's payoffs are non-stationary due to evolving child estimates, the children's drift conditions control this non-stationarity. Theorem 4 then guarantees that the parent's sample averages concentrate around their expected values with high probability.

Together, these ensure that the root node satisfies both components of the drift conditions, possibly with a different constant C_p. Applying Theorems 2 and 5 at the root gives

$$\left|\mathbb{E}[\bar{X}_n] - \mu^*\right| = O\left(\frac{\log n}{n}\right), \qquad \mathbb{P}(\text{select suboptimal root action at time } n) \to 0 \quad \text{at a polynomial rate.}$$

Conclusion. By induction on depth, UCT satisfies the drift conditions at every node of a finite-horizon MDP. Consequently, the bias of the estimated expected payoff is $O(\log n/n)$, and the failure probability at the root decays polynomially.

4.2 Numerical Experiment

I implemented a simple UNO card game to demonstrate the logarithmic decay of the optimal move's bias. Code for this experiment can be found in `uno.py` and `uno-convergence.py`. In my UNO variant, the deck has 40 cards (4 colors $\times$ 10 numbers), and each player starts with 5 cards. On their turn, a player may either draw from the deck or play a card matching the color or number of the top discard. The first to empty their hand wins. When the deck is exhausted, the discard pile is reshuffled into the deck, introducing stochasticity, and players cannot see each other's hands, adding imperfect information.

To improve the quality of the bias-decay plot, I made several implementation decisions. First, I measured bias after $10, 50, 250, 1250, 6250$, and 20000 rollouts, using logarithmically spaced points to capture decay over several orders of magnitude. Computing the exact "true" reward for the optimal move at the root is infeasible, so I approximated it by running 200,000 simulations from the root. This estimate serves as a baseline against which biases for smaller rollout counts are measured. To reduce noise from stochastic initial states, I used a reproducible base state generated with a fixed random seed, ensuring the deck, hands, and initial discard are consistent across runs. In addition, for each rollout count, I ran MCTS 20 times and averaged the results to further minimize stochastic noise. Finally, the theoretical curve $f(x) = \frac{\log(x)}{x}$ was scaled such that its initial point coincides with the first empirical observation. This emphasizes the relative decay rates and the shape of the curve, rather than the absolute magnitudes of the values. Figure 1 shows the empirical bias-decay curve versus the theoretical bound on a log–log scale. The empirical curve initially lies above the theoretical line before aligning with it as the number of rollouts increases. Our empirical bias is the average, over 20 runs, of the difference between the estimated and true payoffs; this only approximates the theoretical quantity

$$\left|\mathbb{E}[X_n] - \mu^*\right|,$$

since both $\mathbb{E}[X_n]$ and μ^* are replaced by finite-sample means and therefore include sampling noise. Early in the search, UCB1's preference for underexplored actions can temporarily inflate value estimates, and combined with rollout noise, may explain the initially elevated bias. For 250 and 1250 rollouts, the standard deviation is notably larger than for smaller or larger rollout counts. This occurs because the search tree is in a transitional state: with very few rollouts (e.g., 10–50), the tree remains largely unexplored, so most simulations traverse the same limited paths, resulting in low variance. At intermediate rollout counts, however, the tree is partially expanded: multiple actions have been explored, but not enough to yield stable value estimates. Suboptimal moves receive enough visits to significantly influence the UCB1 selection of the optimal move, introducing variability in its estimated value. As the rollout count grows large (e.g., 6250+), the law of large numbers takes effect. Each node's value is averaged over many simulations, causing stochastic fluctuations to cancel out and variance to decrease.

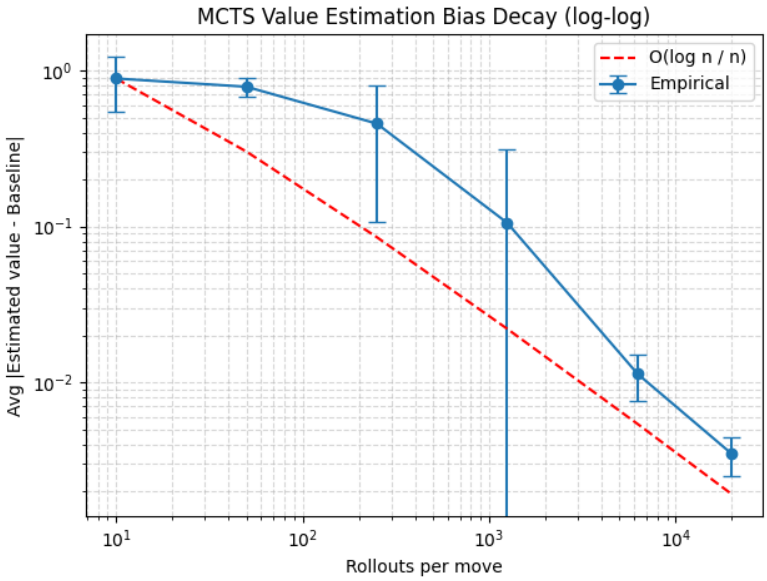

Figure 1: Bias Decay of Optimal Move

5 Tuning Exploration Term

The next question I want to investigate is how tuning the exploration term of UCB1 affects convergence. This question lies at the heart of the exploration-exploitation tradeoff in MCTS. I tuned UCB1 in two settings: UNO and Tic-Tac-Toe. Below are the hyperparameters that I chose to use, in addition to standard UCB1.

U_j	Upper confidence bound for move j
$\bar{X}_j$	Sample mean reward of move j
n	Total simulations from the current node
n_j	Simulations for move j
$\hat{\sigma}_j^2$	Empirical variance of rewards for move j
D	Estimated remaining game length from node
d	Depth of the node in the tree

1. **UCB1-Tuned [1]**:

$$U_j = \bar{X}_j + \sqrt{\frac{\ln n}{n_j} \min\left(\tfrac{1}{4}, \hat{\sigma}_j^2 + \sqrt{\frac{2\ln n}{n_j}}\right)}$$

2. **Depth-scaled UCB1 [3]**:

$$U_j = \bar{X}_j + \left(\frac{\ln n}{n_j}\right)^{\frac{D+d}{2D+d}}$$

In order to implement UCB1-Tuned, I needed to keep track of variance. I found that Welford's online algorithm is a good choice, because it doesn't require maintaining a history of past variances.

$$M_{2,n} = M_{2,n-1} + (x_n - \overline{x_{n-1}})(x_n - \overline{x_n})$$
$$\sigma_n^2 = \frac{M_{2,n}}{n}$$

5.1 UNO Experiment Results

Below are the results from tuning the exploration term in UCB1. Code for this experiment can be found in `uno.py` and `uno-tuning.py`. I tracked the bias decay in the estimated optimal root action as a metric for convergence, similar to the previous section. As Table 1 and Figure 2 show, standard UCB1 and depth-scaled UCB1 significantly outperformed tuned UCB1. After 20,000 iterations, standard UCB1 achieved a bias of 0.00348 and depth-scaled UCB1 achieved 0.00341, while tuned UCB1 remained at 0.35096. All three variants began with high bias, but standard and depth-scaled UCB1 exhibited sharp decay, whereas tuned UCB1's decay was substantially slower. To understand why, consider how each selection policy balances exploration and exploitation:

- **Standard UCB1** prioritizes less-visited nodes, with exploration decaying as the inverse square root of visit count.
- **Depth-scaled UCB1** also prioritizes less-visited nodes, but adjusts the decay rate based on depth: exploration is emphasized more aggressively near the root and reduced near the leaves, matching the intuition that early decisions in the game tree have greater impact.
- **Tuned UCB1** prioritizes less-visited nodes but reduces exploration for actions with low observed variance, based on the principle that low-variance arms require less sampling to estimate accurately.

Since UNO is a highly stochastic, imperfect-information game with a large state space, it benefits from more aggressive exploration than deterministic, perfect-information games with smaller state spaces. From the empirical results, it seems that tuned UCB1 suffers from under-exploration. For instance, actions that initially appear low-variance can lead to widely different outcomes due to hidden information and random draws. Tuned UCB1, which reduces exploration for seemingly low-variance actions, can under-sample promising branches, slowing convergence and resulting in persistently high bias. In contrast, standard and depth-scaled UCB1 maintain aggressive exploration throughout the tree. Depth-scaled UCB1 further improves efficiency by emphasizing exploration near the root, where decisions have the greatest impact on expected outcomes. This combination ensures rapid bias decay.

Table 1: Bias and Standard Deviation by Mode and Sample Size

Mode	**n=10**		**n=50**		**n=250**	
	Avg Bias	Std	Avg Bias	Std	Avg Bias	Std
Standard	0.88957	0.35016	0.78625	0.10821	0.43526	0.32259
Tuned	0.85243	0.35372	0.86907	0.23832	0.57769	0.39314
Depth-scaled	0.85077	0.35389	0.89377	0.25150	0.50286	0.35803
Mode	**n=1250**		**n=6250**		**n=20000**	
	Avg Bias	Std	Avg Bias	Std	Avg Bias	Std
Standard	0.14867	0.26876	0.01136	0.00381	0.00348	0.00099
Tuned	0.49388	0.48059	0.35516	0.46194	0.35096	0.47209
Depth-scaled	0.36181	0.43296	0.06043	0.21263	0.00341	0.00435

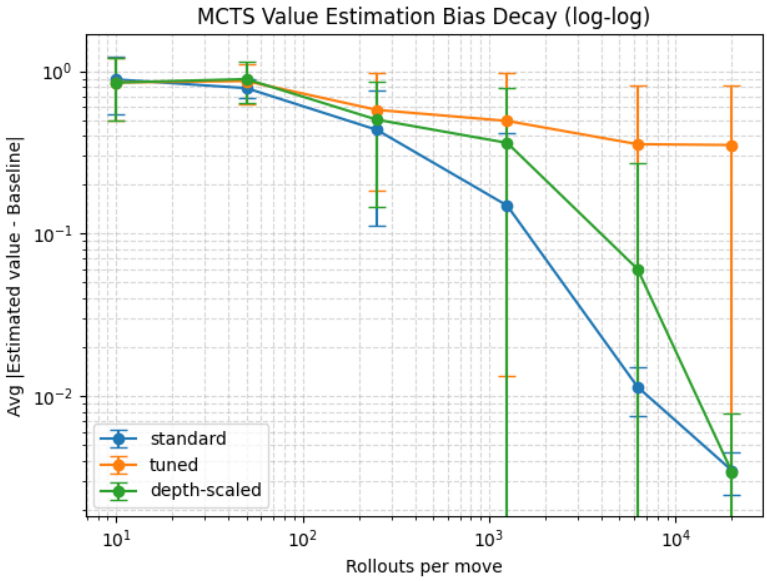

Figure 2: Bias Decay of Optimal Move for Different UCB1 Exploration Terms

5.2 Tic-Tac-Toe Experiment Results

Next, we will discuss the results from the Tic-Tac-Toe experiment. Code for this experiment can be found in `tic-tac-toe-tuning.py`. As before, bias decay in the estimated optimal root action was used as a convergence metric. Interestingly, as Table 2 and Figure 3 show, tuned UCB1 significantly outperformed both standard and depth-scaled UCB1. For low rollout counts, all policies started with low bias, but after 20,000 rollouts, standard UCB1 reached a bias of 0.23782, depth-scaled UCB1 0.20770, and tuned UCB1 only 0.06028, corresponding to the sharpest bias decay. This difference arises from the nature of the game: Tic-Tac-Toe is deterministic, with perfect information and a small state space. In such settings, over-exploration is wasteful, since action outcomes are quickly known. Tuned UCB1 leverages this by reducing exploration for low-variance actions, focusing rollouts on promising moves and achieving rapid convergence. This contrasts with stochastic, large games like UNO, where aggressive exploration is essential to capture the full uncertainty of the game tree.

Table 2: Bias and Standard Deviation by Mode and Sample Size

Mode	**n=10**		**n=50**		**n=250**	
	Avg Bias	Std	Avg Bias	Std	Avg Bias	Std
Standard	0.40000	0.19446	0.35185	0.10030	0.23507	0.04331
Tuned	0.47268	0.10634	0.27307	0.07204	0.25014	0.03376
Depth-scaled	0.44421	0.14444	0.30360	0.06589	0.25482	0.03480
Mode	**n=1250**		**n=6250**		**n=20000**	
	Avg Bias	Std	Avg Bias	Std	Avg Bias	Std
Standard	0.23397	0.02430	0.23618	0.00689	0.23782	0.00156
Tuned	0.23040	0.01973	0.14278	0.01530	0.06028	0.00647
Depth-scaled	0.24304	0.01586	0.24467	0.00748	0.20770	0.00461

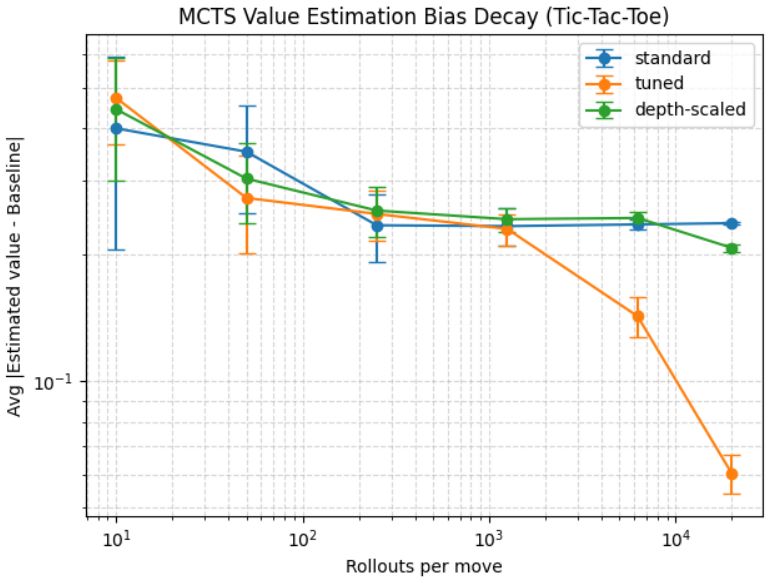

Figure 3: Bias Decay of Optimal Move for Different UCB1 Exploration Terms

6 Finite-Time Regret Analysis in Pathological Tree Construction

Finally, I will study the finite-time regret behavior of UCT in the pathological tree of [2], and compare it against the Flat-UCB variant proposed in the same paper. To do so, I will construct a synthetic example designed to expose the worst-case regret of both algorithms, and conclude with a practical discussion of their contrasting behaviors.

6.1 Pathological Tree Construction

I use the pathological tree from [2], shown in Figure 4. This is a binary tree in which, at every internal node, action 2 immediately terminates with a deceptively high reward, while action 1 transitions to a deeper internal node with no immediate payoff. The terminal rewards of action 2 decrease with depth, whereas repeatedly choosing action 1 eventually reaches a final leaf with the maximal reward of 1. Thus, the best action cannot be recognized until the search has gone all the way to the deepest leaf, since the maximal reward appears only after taking action 1 at every level. For standard UCT, this tree demonstrates hyperexponential regret: discovering the optimal reward can require on the order of $\Omega\Big(\exp(\exp(\cdots\exp(1)\cdots))\Big)$ rounds, where the number of nested exponentials grows with the tree depth.

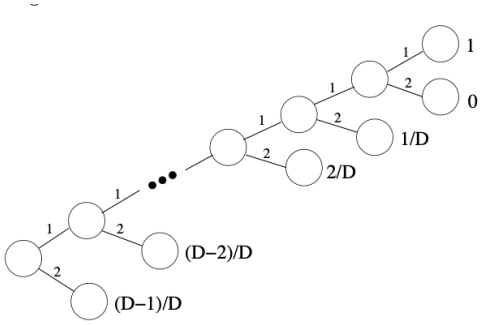

Figure 4: Pathological Tree Construction [2]

6.2 Flat UCB Algorithm

Flat-UCB is a variant of UCT proposed by [2], designed to treat the tree as a bandit over all its leaves. In this formulation, each leaf acts as an arm of a bandit, and each internal node simply inherits the maximal upper confidence bound of its children. All UCBs are initialized to $+\infty$ for unvisited leaves. Formally, for a node v at depth d with n_v visits and total rollouts n, the Flat-UCB bound is defined as

$$\text{bound}(v) = \begin{cases} +\infty, & \text{if } v \text{ is a leaf with } n_v = 0, \\ \hat{\mu}_v + \sqrt{\frac{\ln(1/\beta_n)}{2n_v}}, & \text{if } v \text{ is a leaf with } n_v > 0, \\ \max\big(\text{bound}(v_{\text{left}}), \text{bound}(v_{\text{right}})\big), & \text{if } v \text{ is internal,} \end{cases}$$

where $\hat{\mu}_v$ is the empirical mean reward at leaf v, and the confidence parameter β_n is

$$\beta_n = \frac{\beta}{2^{D+1}\, n\,(n+1)},$$

with D the tree depth and $\beta \in (0,1)$ a global confidence parameter. The parameter $\beta \in (0,1)$ is a user-chosen global confidence level that controls the probability that the Flat-UCB regret bound holds: with probability at least $1-\beta$, the pseudo-regret satisfies

$$\bar{R}_n \le 6 \sum_{i \in S} \frac{1}{\Delta_i} \log\left(\frac{2^{D+2}}{\Delta_i^2 \beta}\right)$$

for all $n \ge 1$ simultaneously [2]. Here, S denotes the set of all suboptimal leaves, $\Delta_i \triangleq \mu^* - \mu_i > 0$ is the suboptimality gap of leaf i, and $\Delta \triangleq \min_{i \in S} \Delta_i$ is the smallest such gap. The quantity $\bar{R}_n \triangleq n\mu^* - \sum_{t=1}^{n} \mu_{I_t}$ is the cumulative pseudo-regret up to horizon n, where I_t is the leaf selected at iteration t.

6.3 Numerical Experiment

Code for this experiment can be found in `regret.py`. I will first describe my implementation choices and then discuss the experimental results. Unlike standard MCTS, I prebuilt the pathological tree using an iterative depth-first search, eliminating the expansion phase. Since all rewards are deterministic, the rollout step was also omitted. To reduce recursion overhead in Flat-UCB, I implemented a function that computes the upper confidence bounds for all nodes via a breadth-first traversal; in each iteration, Flat-UCB simply calls this function and selects the child with the highest bound. Initially, I went for the recursive approach, but the computational overhead led to extreme inefficiencies, even for trees with a depth of 12.

Next, I discuss the experimental results. As shown in Figure 5, after 100,000 iterations, the cumulative regret of Flat-UCB has largely plateaued, indicating that the algorithm has successfully converged to the optimal node. In contrast, the regret for standard UCT continues to grow roughly linearly, suggesting it has not yet identified the optimal path. This behavior can be attributed to the structure of the pathological tree: shallow leaves offer deceptively high rewards, while the optimal branch requires following action 1 repeatedly to reach a deep terminal node with maximal reward. In standard UCT, nodes along this optimal path are visited infrequently, so their exploration bonuses remain low. As a result, the algorithm tends to avoid the optimal branch, creating a "catch-22" in which the algorithm's optimization strategy is designed to avoid the optimal branch. Flat-UCB avoids this problem by forcing at least one visit to every leaf: the upper confidence bound of an internal node is the maximum of its children's bounds, so any unexplored leaf contributes $+\infty$, which propagates to all ancestors and ensures its branch is explored.

Despite its improved worst-case regret, Flat-UCB is not practical, as it requires expanding the entire game tree, undermining the very purpose of MCTS. The value of studying this variant lies instead in revealing where MCTS breaks down and in motivating strategies to mitigate such worst-case behavior.

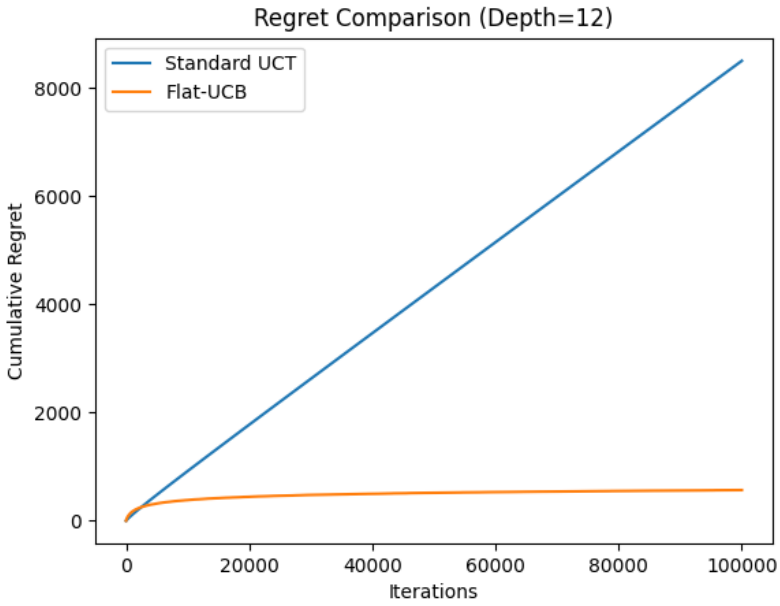

Figure 5: Finite Time Regret Comparison Between UCT and Flat UCB

7 Conclusion

In this report, I reviewed foundational literature on Monte Carlo Tree Search to trace its origins in bandit algorithms, develop an intuition for the exploration–exploitation trade-off, and examine its limitations in worst-case regret scenarios. I implemented numerical simulations to concretize these theoretical insights. While modern applications of MCTS often combine it with deep neural networks, where the neural networks tend to dominate the learning process, studying the original MCTS provides a valuable perspective on the principles that underpin modern reinforcement learning.

References

[1] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2):235–256, May 2002.

[2] Pierre-Arnaud Coquelin and Rémi Munos. Bandit algorithms for tree search. In *Proceedings of the Twenty-Third Conference on Uncertainty in Artificial Intelligence (UAI'07)*, pages 67–74. AUAI Press, 2007.

[3] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In Johannes Fürnkranz, Tobias Scheffer, and Myra Spiliopoulou, editors, *Machine Learning: ECML 2006*, pages 282–293, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg.